Course Name : Fundamentals of Computer and Programming

Project Title : The Knight's Tour Problem

Student Name : Saleh Zabihipazhouh

Student ID : 40334184

Submission Date : 1403/11/12

---

1. Introduction

1.1. Problem Statement

We have 6 X 6 chess board and a knight. Knight can start every square and we have to make program to say if this problem has solution display a 6 X 6 matrix with the count move otherwise say solution not found

1.2 Objective

- Understanding the Knight's movement

- Exploring Hamiltonian paths

- Implementing Backtracking

- Solving a real world problem

- Optimizing the solution

- Enhancing problem solving skills

- Visualizing the solution

- Exploring variations of problem

- Learning recursion

- Building a foundation for advance algorithms

---

2. Methodology

2.1. Algorithms Description

- Backtracking Algorithm:

- The backtracking algorithm is the most common approach to solve the Knight's Tour problem.it systematically explores all possible moves and backtrack when a dead end is reached.

2.2. Implementations Details

- Use the Python language and use PyCharm text editor and File Handling and don't use any libraries.

- Chessboard Size, Starting Position, Knight Moves, Unvisited Squares, Single Solution.

---

3. Function Documentation

1. valid_move(x,y,board):

Purpose:

- Check the move is valid or not.

Parameters:

- X: An int in x axis and show the line of knight location.
- Y: An int in y axis and show the column of knight location.
- Board: A list of list with the 6 length each list show the 2D chess board.

Return Value:

- A bool type True if the move valid and False if the move isn't valid.

Logic:

- The position is within the bounds of chessboard.
- The position hasn't been visited before.

2. print_board(board):

Purpose:

- Print the current state of chessboard.

Parameters:

- Board (list[list[int]]): The chessboard represented as a 2D list.

Return:

- None

Logic:

- This function prints the chessboard in readable format. Each cell is displayed with a width of 2 characters, ensuring proper alignment.

3. solve_problem(x, y, move_count, board):

Purpose:

- Solves the Knight's Tour problem using backtracking algorithm.

Parameters:

- X (int): the current row index of the knight.
- Y (int): the current column index of the knight.
- Move_count (int): the current move number (starting from 1 to 36).
- Board (list[list[int]]): the chessboard represented as a 2D list.

Returns:

- Bool : True if a solution is found, False otherwise.

Logic:

- Marks the current position as visited by setting board[x][y] = move_count
- Checks if all squares have been visited (move_count == n * n). If so it print solution and return True.
- Attempts all possible knight moves from current position. For each valid move, it recursively calls itself.
- If no valid moves lead to a solution, it backtracks by marking the current position as unvisited (board[x][y] = -1) and return False.

4. knight_tour():

Purpose:

- Initializes the chessboard and starts the Knight's Tour from a specified position.

Parameters:

- None

Return:

- None

Logic:

- Initializes the chessboard as a 2D list with all squares marked as unvisited(-1).
- Starts the Knight's Tour from every position.

- Calls the solve_problem function to find a solution. If no solution is found, it prints "No solution".

---

4. Input and Output

4.1. Input

- This code is don't need input parameter.

4.2. Output

- If solution found print it, print "No solution found" otherwise.

---

5. Conclusion

- The Knight's Tour problem successfully demonstrated the application of backtracking to solve a classic combinational problem. By implementing a solution for a 6x6 chessboard, we gained valuable insights into algorithmic thinking, problem solving, and the importance of optimization. While the backtracking approach worked effectively for smaller boards, its exponential time complexity highlighted the need for further improvements, such as Warnsdorff's rule. This project not only reinforced fundamental programming concepts but also provided a foundation for exploring more advanced algorithms and real world applications. Overall, it was an enriching experience that enhanced both technical and analytical skills.